claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-a7b517b83881c74b6.jsonl`
- SHA-256: `977cfb87b3b873a213731884edaa4dcd6421fa5f6a5a161cb1264cb88d6439a3`
- Source modified: `2026-07-08T06:34:40+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:26:58.619Z)

```
You are reviewing an UNCOMMITTED diff in an isolated git worktree at
C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on
origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and
`git --no-pager diff --stat`; read the new files
docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly.
Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-
plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology
question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-
onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts
the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata
md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched
clip, gated by deployment.trust_submit_video_id (default True) and guarded by
orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is
deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4
(phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design.
For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say
so explicitly.
DIMENSION: CORRECTNESS of win #2 (the trust path + wiring). Scrutinize:
- backend/orchestration.py _execute_processing: the trusted-vs-rehash branch inside the
try/except. Does the exception-cleanup (cleanup_acquired_local_input) still fire correctly when
the trust path (no compute_video_id call) is taken? Any path where local_video is set but
video_id assignment throws?
- _looks_like_video_id: correct for all inputs (None, int, empty, 31/32/33 chars, upper/lower,
non-hex)? Does it match what submit_time_video_id actually returns (check
backend/storage/gcs.py::_md5_hex ? base64?hex, lowercase, len 16 bytes)?
- backend/api/job_worker.py: trusted_video_id=job.get("video_id") or None ? is job['video_id']
the right source? Could it ever be a NON-authoritative value (e.g. set from a client-supplied
field somewhere)? Trace submit_process_job / claim_or_create_process_job to confirm
jobs.video_id is only ever set from submit_time_video_id (server-derived), never from the
client's /api/process body.
- Interaction with _maybe_dispatch_remote's divergence guard (result.video_id != video_id, ~line
661): does trusting the submit-time id introduce spurious divergence warnings or mask a real
one?
- Interaction with the #518 validation-verdict cache and probe_process_cache: does the video_id
value change in any case vs before (it must be byte-identical)?
- Does the CLI path (backend/main.py process_video CLI) or any other _execute_processing caller
regress? (Only job_worker passes trusted_video_id; all others default None ? re-hash.)
```

2. assistant (2026-07-08T06:27:01.078Z)

```
I'll start by examining the diff and the relevant files in the worktree.
```

3. user (2026-07-08T06:27:03.651Z)

```
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the
next time Git touches it
 backend/api/job_worker.py                          |   5 +
 backend/config/models.py                           |  10 +
 backend/orchestration.py                           |  51 ++++-
 .../L4_PRIMARY_TOPOLOGY.md                         | 225 ++++++++++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   2 +
 tests/test_config_deployment.py                    |   2 +
 tests/test_video_id_trust.py                       | 173 +++++++++++++++
 7 files changed, 466 insertions(+), 2 deletions(-)
```

4. user (2026-07-08T06:27:10.044Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
                 db,
                 req,
                 progress=lambda stage: db.set_job_progress(job_id, stage),
+                # #524 win #2: hand the drain the submit-time md5 already on the job row
+                # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
+                # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+                # (local upload / non-gcs URI) ? the drain hashes as before.
+                trusted_video_id=job.get("video_id") or None,
             )
         if result.get("video_id"):
             db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win
#2).
+    # The chunked upload already streams the whole-file md5 (``backend/api/uploads.py``), and
the
+    # submit hook records it on the job row (``submit_time_video_id``, one gs:// metadata stat
? #484);
+    # that value is BYTE-IDENTICAL to ``compute_video_id`` (same bytes, same MD5) and is
ALREADY
+    # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True (default)
the drain
+    # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
+    # (a local/appliance upload or a non-gcs direct-URI submit ? ``submit_time_video_id``
returns None)
+    # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+    # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+    trust_submit_video_id: bool = True


 class BillingConfig(BaseModel):
diff --git a/backend/orchestration.py b/backend/orchestration.py
```

```
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
         return None


+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
+    ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust
path:
+    a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
would
+    have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
+    base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
+
 def _cached_process_payload(
     video_id: str, validation_results, segments, requested_compute
 ) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(


 def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
 ) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
     test code that calls ``_execute_processing(req)`` still works.

     ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
     Raises on unexpected internal errors ? the caller records those as a job failure
     """
     import backend.main as _main  # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
         local_video = storage.acquire_local_input(
             req.video_path, getattr(config, "storage", None)
         )
-        video_id = _main.compute_video_id(local_video)
+        # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+        # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+        # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+        # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
```

```
+        # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+        # is unchanged; only the redundant hash is skipped.
+        trust_submitted = bool(
+            getattr(
+                getattr(config, "deployment", None), "trust_submit_video_id", True
+            )
+        )
+        # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+        # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+        # silently coerce it into the DB key.
+        candidate = trusted_video_id or ""
+        if trust_submitted and _looks_like_video_id(candidate):
+            video_id = candidate
+            logger.info(
+                "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+                video_id,
+            )
+        else:
+            video_id = _main.compute_video_id(local_video)
     except Exception:
         storage.cleanup_acquired_local_input(
             req.video_path, local_video, getattr(config, "storage", None)
```

5. assistant (2026-07-08T06:27:12.446Z)

Let me read the full `_execute_processing` function and surrounding code.

6. user (2026-07-08T06:27:13.630Z)

```
720
721
722     def _execute_processing(
723         config=None, db=None, req=None, progress=None, trusted_video_id=None
724     ) -> Dict[str, Any]:
725         """The full hash ? validate ? segment ? report pipeline for one video.
726
727         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
728         module-level ``global_config``/``db_instance`` (backward-compatible with existing
729         tests that pass only ``req``). Route handlers always pass explicit params; older
730         test code that calls ``_execute_processing(req)`` still works.
731
732         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
733
734         ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows
for this
735         job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
736         ``compute_video_id`` and already trusted for the submit-time cache probe. When
present (and
737         ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-
hashing the
738         multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes
as before.
739         It is a keyword-only, internal channel ? NOT a client-supplied request field (a
caller's
740         ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative
id).
741
742         Raises on unexpected internal errors ? the caller records those as a job failure
743         """
744         import backend.main as _main  # call-time seam: main's globals/model stay patchable
745
```

```
        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = ""
        try:
            local_video = storage.acquire_local_input(
                req.video_path, getattr(config, "storage", None)
            )
            # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
the whole
            # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
            # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
submit-time
            # cache probe trusts, so reusing it here is consistent, not a new trust
assumption. Gated by
            # deployment.trust_submit_video_id (default on) and only honoured when the id is
well-formed
            # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
            # is unchanged; only the redundant hash is skipped.
            trust_submitted = bool(
                getattr(
                    getattr(config, "deployment", None), "trust_submit_video_id", True
                )
            )
            # STRICT: trust only an id in the EXACT form the drain would have produced
(32-char
            # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
? a
            # differently-cased or malformed id is a signal something is off, so re-hash
rather than
            # silently coerce it into the DB key.
            candidate = trusted_video_id or ""
            if trust_submitted and _looks_like_video_id(candidate):
                video_id = candidate
                logger.info(
                    "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
                    video_id,
                )
            else:
```

```
798                 video_id = _main.compute_video_id(local_video)
799         except Exception:
800             storage.cleanup_acquired_local_input(
801                 req.video_path, local_video, getattr(config, "storage", None)
802             )
803             raise
804         try:
805             existing_video = db.get_video(video_id)
806         except Exception:
807             storage.cleanup_acquired_local_input(
808                 req.video_path, local_video, getattr(config, "storage", None)
809             )
810             raise
811         was_reingest = existing_video is not None
812 
813         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
814         default_seg = getattr(
815             getattr(config, "indexing", None), "default_segmenter", "motion"
816         )
817         seg_name = req.segmenter or default_seg
818         segmenter_actual = None
819         segmentation_ran = False
820         val_results: List[Any] = []
821         val_context: Dict[str, Any] = {}
822         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
823         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
824         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
825         compute_actual: Optional[str] = None
826         response: Optional[Dict[str, Any]] = None
827         try:
828             if (
829                 existing_video
830                 and existing_video.get("status") == "processed"
831                 and not req.force
832             ):
833                 cached_segments = db.query_play_segments(video_id, {})
834                 if cached_segments:
835                     logger.info(
836                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
837                         video_id,
838                     )
839                     response = _cached_process_payload(
840                         video_id,
841                         existing_video.get("validation_results", []),
842                         cached_segments,
843                         getattr(req, "compute", None),
844                     )
845                     return response
846 
847             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
848             #    UNLESS this exact content already has a cached verdict under the current
validator config.
849             notify("validating")
850             # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
851             # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
852             # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
853             # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
```

```
which the
854             # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
855             # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
856             # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
857             # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
858             # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
859             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
```

7. assistant (2026-07-08T06:27:23.814Z)

Now let me examine the gcs `_md5_hex` and the submit-time / job creation path.

8. user (2026-07-08T06:27:25.955Z)

```
24:def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25:    """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB key)
27:    have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28:    if not md5_b64:
31:        raw = base64.b64decode(md5_b64, validate=True)
83:            md5=_md5_hex(getattr(b, "md5_hash", None)),
```

9. user (2026-07-08T06:27:29.168Z)

```
24      def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25          """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26          is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27          have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28          if not md5_b64:
29              return None
30          try:
31              raw = base64.b64decode(md5_b64, validate=True)
32          except (binascii.Error, ValueError):
33              return None
34          return raw.hex() if len(raw) == 16 else None
35
36
37      class GCSStorage(StorageBackend):
38          def __init__(
39              self,
40              *,
41              client: Any = None,
42              project: Optional[str] = None,
43              signed_url_ttl: int = 3600,
44              **_ignored,
45          ) -> None:
46              self._ttl = signed_url_ttl
47              if client is not None:
48                  self._client = client
49                  return
50              storage_module: Any = import_module("google.cloud.storage")
51              self._client = storage_module.Client(project=project)
52
53          def _blob(self, ref: StorageRef):
54              return self._client.bucket(ref.bucket).blob(ref.key)
55
56          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
```

```
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os

            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
            return ref

        def exists(self, ref) -> bool:
            return bool(self._blob(ref).exists())

        def stat(self, ref) -> StorageStat:
            b = self._blob(ref)
            b.reload()
            return StorageStat(
                size=int(b.size or 0),
                modified=b.updated,
                etag=b.etag,
                content_type=b.content_type,
                md5=_md5_hex(getattr(b, "md5_hash", None)),
```

10. user (2026-07-08T06:27:35.696Z)

```
"""Canonical file-identity hashing for the indexer.

`video_id` is the MD5 of the *whole* video file. Historically each entrypoint
(`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
any future change to one copy would silently diverge the DB key.

This module is the single source of truth. Import `compute_video_id` everywhere a
video needs to be keyed.
"""

from __future__ import annotations

import hashlib
```

64 KB matches the value used by the live pipeline (backend/main.py) before this was centralized. The chunk size does not affect the resulting digest ? it only bounds memory use while streaming large files.

```
_CHUNK_SIZE = 65536


def compute_video_id(filepath: str) -> str:
    """Return the MD5 hex digest of the whole file ? the canonical video_id.

    Streams the file in fixed-size chunks so multi-gigabyte recordings never load
    into memory at once.
    """
    hasher = hashlib.md5()
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
            hasher.update(chunk)
```

```
    return hasher.hexdigest()
```

11. assistant (2026-07-08T06:27:39.256Z)

Now let me trace the job creation path to confirm `jobs.video_id` is only set from server-derived `submit_time_video_id`.

12. user (2026-07-08T06:27:41.732Z)

```
30:    db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
33:    ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
38:    hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
41:    claimed = db.claim_or_create_process_job(
51:        video_id=video_id,
74:    from backend import orchestration  # call-time seam (mirrors submit_process_job)
80:def submit_process_job(
92:       per video_id) and completes the job instantly: no drain, no GPU dispatch.
98:    video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99:    probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100:        db, cfg, video_id
104:    outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
113:            db, cfg, video_id, req.video_path, requested_compute=req.compute
185:                "video_id": j.get("video_id"),
```

13. user (2026-07-08T06:27:44.682Z)

```
28
29      def claim_process_job(
30          db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31      ) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32          """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33          ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34          double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
35          ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
36          just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
37          ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
38          hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39          the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40          can re-associate the job with its bucket outputs (CP-rebuild §4.2); "" ? the drain
fills it."""
41          claimed = db.claim_or_create_process_job(
42              job_id,
43              req.video_path,
44              segmenter=req.segmenter,
45              player_pool=req.player_pool,
46              max_frames=req.max_frames,
47              owner_id=owner_id,
48              locale=req.locale,
49              compute=req.compute,
50              force=req.force,
51              video_id=video_id,
52          )
53          if claimed is None:
54              return None
```

```
        if claimed["created"]:
            return (True, None)
        return (
            False,
            JSONResponse(
                status_code=202,
                content={
                    "job_id": claimed["job_id"],
                    "status": claimed["status"],
                    "poll": f"/api/jobs/{claimed['job_id']}",
                    "deduplicated": True,
                },
            ),
        )


    def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:
        """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
        (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
copy."""
        from backend import orchestration  # call-time seam (mirrors submit_process_job)

        need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
        require_free_bytes(tempfile.gettempdir(), need, stage="fetch")


    def submit_process_job(
        db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
    ) -> Optional[Tuple[bool, JSONResponse]]:
        """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
one hook.

        Order matters (#492 review):
        1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
effects.
        2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
           fetches to local scratch, so low temp space must not reject it (review blocker
2).
        3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
           re-hydrate side effect, so the first request owns the cache completion (blocker
1).
        4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
serialized
           per video_id) and completes the job instantly: no drain, no GPU dispatch.

        Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
``None`` ?
        DB failure (the caller 500s)."""
        from backend import orchestration  # call-time seam: tests patch orchestration attrs

        video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
        probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
            db, cfg, video_id
        )
        if input_ref.backend != "local" and not probed:
            _fetch_scratch_guard(cfg, input_ref)  # raises 507 pre-claim, exactly as before
        outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
        if outcome is None:
            return None
        created, dedup_response = outcome
        if not created:
            assert dedup_response is not None  # claim contract: not-created ? dedup
response
            return (False, dedup_response)
```

```
        if probed:
            cached = orchestration.cached_process_result(
                db, cfg, video_id, req.video_path, requested_compute=req.compute
            )
            if cached is not None:
                db.mark_job_done(job_id, cached)
                return (
                    False,
                    JSONResponse(
                        status_code=202,
                        content={
                            "job_id": job_id,
                            "status": "done",
                            "poll": f"/api/jobs/{job_id}",
                            "cached": True,
                        },
                    ),
                )
            # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
            # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
            # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
            # (a bare raise would leave a phantom 'queued' job for the drain to grab).
            if input_ref.backend != "local":
                try:
                    _fetch_scratch_guard(cfg, input_ref)
                except HTTPException:
                    db.mark_job_failed(
                        job_id,
                        "cache re-hydrate failed and local scratch cannot hold a fetch of "
                        "the source; free disk space or retry later",
                    )
                    raise
        return (
            True,
            JSONResponse(
                status_code=202,
                content={
                    "job_id": job_id,
                    "status": "queued",
                    "poll": f"/api/jobs/{job_id}",
                },
            ),
        )

    # Execution states a job can be in. ACTIVE = still doing (or about to do) work; the
queue view defaults
    # to these. done/failed are terminal and only surface under ?status=all (or an explicit
single status).
    ACTIVE_JOB_STATES = ("queued", "running", "waiting")
```

14. user (2026-07-08T06:27:50.771Z)

```
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(
            os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
        )
        out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        return bool(has_job and out_root)


    def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
```

```
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236                 getattr(input_ref, "uri", input_ref),
237                 exc_info=True,
238             )
239             return None
240
241
242     def _looks_like_video_id(value: Optional[str]) -> bool:
243         """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5
that
244         ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2
trust path:
245         a trusted id is only reused in place of a re-hash when it has the EXACT shape the
drain would
246         have computed, so a malformed / truncated / wrong-encoding id can never become the
DB key ?
247         it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the
gs://
248         base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
249         return (
250             isinstance(value, str)
251             and len(value) == 32
252             and all(c in "0123456789abcdef" for c in value)
253         )
254
```

15. user (2026-07-08T06:27:58.372Z)

```
94:from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
258:class ProcessRequest(BaseModel):
277:    # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
verbatim.
278:    video_id: str
289:    # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
297:    video_id: str
```

```
305:    video_id: str
439:    "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
448:        "video_id": row.get("id"),
461:    (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
482:    does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
512:    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
549:def _source_cache_lock(video_id: str) -> threading.Lock:
551:        return _source_cache_locks.setdefault(video_id, threading.Lock())
554:def _source_cache_path(video_id: str, input_ref, cfg) -> str:
559:    name = safe_output_filename(f"{video_id}{ext}")
563:def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:    cache_path = _source_cache_path(video_id, input_ref, cfg)
568:    lock = _source_cache_lock(video_id)
591:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
596:        if video_id in _proxy_inflight:
598:        _proxy_inflight.add(video_id)
605:            src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608:        logger.warning("proxy: warm failed for %s: %s", video_id, e)
611:            _proxy_inflight.discard(video_id)
614:@app.get("/api/source/{video_id}")
615:def get_source(video_id: str, request: Request):
621:    rec = request.app.state.db.get_video(video_id)
628:        proxy_name = safe_output_filename(f"{video_id}.mp4")
655:            local = _cached_remote_source(video_id, input_ref, cfg)
667:        target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
677:    ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?
so it joins
681:    Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
691:        name = safe_output_filename(f"{req.video_id}.rallies.csv")
693:        raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
707:        "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
720:def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721:    """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``."""
722:    return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
753:    MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
```

16. user (2026-07-08T06:28:02.558Z)

```
258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269         compute: Optional[str] = None
270         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
```

```
272         force: bool = False
273
274
275     class AnnotationsRequest(BaseModel):
276         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278         video_id: str
279         csv: str
280
281
282     class AssetRequest(BaseModel):
283         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285         #                   local working copy for the run), OR
286         #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
287         #                   registered as-is, no copy (D14b).
288         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
289         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290         uploaded_path: Optional[str] = None
291         source_uri: Optional[str] = None
292         medium_uri: str
293         sport: Optional[str] = None
294
295
296     class QueryRequest(BaseModel):
297         video_id: str
```

17. user (2026-07-08T06:28:14.488Z)

```
backend/api/jobs.py:29:def claim_process_job(
backend/api/jobs.py:33:    ``db.claim_or_create_process_job`` (ONE write-locked op) so two
concurrent identical submits ? a
backend/api/jobs.py:41:    claimed = db.claim_or_create_process_job(
backend/api/jobs.py:74:    from backend import orchestration  # call-time seam (mirrors
submit_process_job)
backend/api/jobs.py:80:def submit_process_job(
backend/api/jobs.py:104:    outcome = claim_process_job(db, job_id, req, owner_id,
video_id=video_id)
backend/infrastructure/database_jobs.py:120:    def set_job_video_id(self, job_id: str,
video_id: str) -> bool:
backend/infrastructure/database_jobs.py:228:    def claim_or_create_process_job(
backend/main.py:950:    # The P2 (D11) remote-source scratch guard now lives INSIDE
jobs.submit_process_job, AFTER
backend/main.py:964:    outcome = jobs_api.submit_process_job(
```

18. user (2026-07-08T06:28:17.830Z)

```
228         def claim_or_create_process_job(
229             self,
230             job_id: str,
231             video_path: str,
232             segmenter: Optional[str] = None,
233             player_pool: Optional[List[str]] = None,
234             max_frames: Optional[int] = None,
235             policy: Optional[Dict[str, Any]] = None,
236             owner_id: Optional[str] = None,
237             locale: Optional[str] = None,
238             compute: Optional[str] = None,
```

```
239             force: bool = False,
240             video_id: str = "",
241         ) -> Optional[Dict[str, Any]]:
242             """ATOMIC idempotent submit for a PROCESS job (fixes the read-before-insert race
? PR #481
243             review): in ONE write-locked statement, insert ``job_id`` as 'queued' ONLY IF no
active
244             (queued|running|waiting) process job already exists for (owner_id, video_path).
Two concurrent
245             identical submits (a double-click / retry) therefore can't both insert ? SQLite
serializes
246             writers, and the second's ``INSERT ? WHERE NOT EXISTS`` sees the first's row
(same
247             compare-and-set idiom as ``claim_due_job``, no read-then-write gap). ``force``
skips the guard
248             (deliberate reprocess ? always inserts). Returns
``{"job_id","status","created"}`` ? ``created``
249             False means an active job already existed and the caller must NOT dispatch a
second run ? or
250             ``None`` on failure.
251
252             ``video_id`` is the submit-time MD5 when the caller could resolve one cheaply
(#484 ?
253             gs:// object metadata); "" ? unknown-yet (the drain fills it after hashing, as
before).
254             Persisting it at CREATE is what lets a restarted control plane re-associate a
running
255             remote job with its bucket outputs (CONTROL_PLANE_DURABLE_REBUILD §4.2)."""
256             params = json.dumps({"force": True}) if force else None
257             vals = (
258                 job_id,
259                 video_path,
260                 video_id,
261                 segmenter,
262                 json.dumps(player_pool) if player_pool is not None else None,
263                 max_frames,
264                 json.dumps(policy) if policy is not None else None,
265                 owner_id,
266                 locale,
267                 compute,
268                 params,
269             )
270             owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
271             guard = () if owner_id is None else (owner_id,)
272             cols = (
273                 "id, video_path, video_id, segmenter, player_pool, max_frames, status,
policy, "
274                 "owner_id, locale, compute, params"
275             )
276             try:
277                 with self._connect() as conn:
278                     cur = conn.cursor()
279                     if force:
280                         cur.execute(
281                             f"INSERT INTO jobs ({cols}) VALUES (?, ?, ?, ?, ?, ?, 'queued',
?, ?, ?, ?, ?)",
282                             vals,
283                         )
284                         conn.commit()
285                         return {"job_id": job_id, "status": "queued", "created": True}
286                     # Conditional insert ? one atomic statement: only if no active process
job exists for
287                     # this (owner, input). SQLite serializes writers, so a racing second
submit sees the row.
288                     cur.execute(
```

```
289                         f"INSERT INTO jobs ({cols}) "
290                         "SELECT ?, ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS
("
291                         "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
292                         "AND status IN ('queued', 'running', 'waiting') "
293                         "AND (kind IS NULL OR kind != 'compile'))",
294                         vals + guard + (video_path,),
295                     )
296                     conn.commit()
297                     if cur.rowcount == 1:
```

19. user (2026-07-08T06:28:23.906Z)

```
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:
163                 result = _main._execute_compile(
164                     config,
165                     db,
166                     job,
167                     progress=lambda stage: db.set_job_progress(job_id, stage),
168                 )
169             else:
170                 req = _main._job_to_request(job)
171                 result = _main._execute_processing(
172                     config,
173                     db,
```

```
174                     req,
175                     progress=lambda stage: db.set_job_progress(job_id, stage),
176                     # #524 win #2: hand the drain the submit-time md5 already on the job row
177                     # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
178                     # compute_video_id) so it can skip re-hashing the fetched clip. "" ?
unknown
179                     # (local upload / non-gcs URI) ? the drain hashes as before.
180                     trusted_video_id=job.get("video_id") or None,
181                 )
182             if result.get("video_id"):
183                 db.set_job_video_id(job_id, result["video_id"])
184             db.mark_job_done(job_id, result)
185             if not is_compile:
186                 _maybe_record_job_cost(
187                     job, result, config, db
188                 )  # M8 cost ledger (default-OFF; process jobs only)
189                 _maybe_run_flywheel(
```

20. user (2026-07-08T06:28:30.552Z)

```
65:        video_id: str,
75:        NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
85:             "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
87:             (job_id, video_path, video_id, owner_id, locale, params),
98:                    "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
120:    def set_job_video_id(self, job_id: str, video_id: str) -> bool:
122:             "UPDATE jobs SET video_id=? WHERE id = ?",
123:             (video_id, job_id),
124:             f"Failed to set video_id for job {job_id}",
151:                 .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
184:             "SELECT id, kind, status, progress, video_id, video_path, compute, error,
result, "
217:                     "SELECT id, status FROM jobs WHERE "
240:        video_id: str = "",
252:        ``video_id`` is the submit-time MD5 when the caller could resolve one cheaply (#484
?
260:             video_id,
273:             "id, video_path, video_id, segmenter, player_pool, max_frames, status, policy, "
290:                     "SELECT ?, ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS ("
291:                     "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
302:                         "SELECT id, status FROM jobs WHERE " + owner_clause + " AND
video_path = ? "
328:                     "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
418:                     "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
432:    def get_video_cost(self, video_id: str) -> Dict[str, Any]:
439:                     "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
440:                     "WHERE video_id = ?",
441:                     (video_id,),
461:        return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
495:                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
508:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
512:        first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds nothing
516:        can SELECT the same head-of-queue row, and if the losers reported None the caller
522:        the SELECT is an autocommit read and each CAS commits before the next iteration, so
no
529:                         "SELECT id, status FROM jobs WHERE status='queued' "
543:                     # Lost the race: the winner flipped this row; re-SELECT for the next
one.
582:        job whose ``video_id`` is already known (CP-rebuild P3): its remote worker may still
be
616:                     "AND NOT (status = 'running' AND video_id IS NOT NULL AND video_id != ''
```

```
"
636:                    "SELECT id, video_id, video_path, owner_id, compute FROM jobs "
638:                    "AND video_id IS NOT NULL AND video_id != '' "
```

21. user (2026-07-08T06:28:33.827Z)

```
508         def claim_due_job(self) -> Optional[Dict[str, Any]]:
509             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
510             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
511             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
512             first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds
nothing
513             runnable ? never for a lost race.
514
515             A lost CAS RETRIES instead of returning: under the parallel drain (#466) several
workers
516             can SELECT the same head-of-queue row, and if the losers reported None the
caller
517             (``_drain_due_jobs``) would read it as "queue empty" and stop with runnable jobs
still
518             queued ? stranding dispatch capacity until some later submit/wake. The retry
loop is
519             deliberately UNBOUNDED: every lost race means another worker successfully
claimed that
520             row, so the runnable set shrank and global progress is guaranteed (lock-free
progress);
521             a retry cap would just reintroduce the false-empty. One connection serves all
attempts ?
522             the SELECT is an autocommit read and each CAS commits before the next iteration,
so no
523             write lock is ever held across a retry (busy_timeout covers momentary
contention)."""
524             try:
525                 with self._connect() as conn:
526                     cur = conn.cursor()
527                     while True:
528                         row = cur.execute(
529                             "SELECT id, status FROM jobs WHERE status='queued' "
530                             "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at
<= CURRENT_TIMESTAMP)) "
531                             "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
532                         ).fetchone()
533                         if not row:
534                             return None  # nothing runnable ? a TRUE empty, not a lost race
535                         cur.execute(
536                             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP
"
537                             "WHERE id=? AND status=?",
538                             (row["id"], row["status"]),
539                         )
540                         conn.commit()
541                         if cur.rowcount == 1:
542                             break  # claimed it
543                         # Lost the race: the winner flipped this row; re-SELECT for the next
one.
544                 return self.get_job(row["id"])
545             except Exception as e:
546                 logger.error(f"Failed to claim a due job: {e}")
547                 return None
548
549         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
550             """Crash recovery: any job still queued/running from a previous process is dead
```

```
551             (the executor is in-process). Mark failed so clients see a terminal state, not a
552             hang. Returns the number of jobs swept.
553
554             NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
555             ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
This
556             blanket-fail method stays for callers/tests that want the old behaviour."""
557             try:
558                 with self._connect() as conn:
559                     cur = conn.cursor()
560                     cur.execute(
561                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
562                         "WHERE status IN ('queued', 'running')",
563                         (reason,),
564                     )
565                     conn.commit()
566                     return cur.rowcount
567             except Exception as e:
568                 logger.error(f"Failed to sweep stale jobs: {e}")
569                 return 0
570
571         def recover_stale_jobs(
572             self, reason: str = "interrupted by server restart"
573         ) -> Tuple[int, int]:
574             """Crash recovery, kind-aware (#329 serving reliability). Called once at
startup.
575
576             A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
the same
577             segments yields the same reel), so a compile left queued/running by a killed
instance is
```

22. assistant (2026-07-08T06:28:43.290Z)

Now let me examine the divergence guard in `_maybe_dispatch_remote`.

23. user (2026-07-08T06:28:44.297Z)

```
620         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
621         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
622         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
623         wasb_cfg = getattr(idx_cfg, "wasb", None)
624         if wasb_cfg is not None:
625             decode_backend = getattr(wasb_cfg, "decode_backend", None)
626             if decode_backend:
627                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
628             batch_size = getattr(wasb_cfg, "batch_size", None)
629             if batch_size:
630                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
631             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
632             if prefetch:
633                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
634         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
635         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
636         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
637         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
638         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
```

```
min_blur_threshold=8.0,
639         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
640         spec = ComputeJobSpec(
641             video_uri=req.video_path,
642             out_uri=out_root,
643             segmenter=seg_name,
644             config_overrides=tuple(overrides),
645             skip_validation=True,
646         )
647         try:
648             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
649             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
650             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
651             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
652             result = compute.run_infer(
653                 spec,
654                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
655             )
656         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
657             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
658             return _failed(f"cloud dispatch error: {e}", True)
659         if result.returncode != 0:
660             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
661         if result.video_id and result.video_id != video_id:
662             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
663             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
664             logger.warning(
665                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
666                 "and the worker hash; ingesting under the local id.",
667                 result.video_id,
668                 video_id,
669             )
670
671         notify("ingesting (cloud_run)")
672         storage_dict = (
673             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
674             if hasattr(storage_cfg, "model_dump")
675             else {}
676         )
677         try:
678             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
679         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
680             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
681             return _failed(
682                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
683                 True,
684             )
685
686         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
687         segments = [
688             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
689         ]
690         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
691         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
692         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
```

```
<execution>).
693         provenance = {
694             "path": "cloud_run",
695             "requested": req.compute,
696             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
697             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
698             "execution": result.execution,
699             "outputs_uri": result.outputs_uri,
700         }
701         logger.info(
702             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
703             result.execution,
704             provenance["job"],
705             provenance["region"],
706             result.outputs_uri,
707             video_id,
708         )
709         return (
710             {
711                 "video_id": video_id,
712                 "status": "success",
713                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
714                 "results": [r.model_dump() for r in val_results],
715                 "play_segments": segments,
716                 "compute": provenance,
717             },
718             True,
719         )
```

24. assistant (2026-07-08T06:29:24.930Z)

Let me read the remainder of `_execute_processing` to verify end-of-function cleanup.

25. user (2026-07-08T06:29:26.452Z)

```
859             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
860             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
861             val_fingerprint = registry.config_fingerprint(config)
862             # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
863             # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
864             # failing the job. The cache is a pure speedup ? never a new failure mode.
865             reused_validation = False
866             if not req.force:
867                 try:
868                     cached = db.get_validation_cache(video_id)
869                     if cached is not None and cached.get("fingerprint") == val_fingerprint:
870                         # Replay the stored ValidationResult list; val_context stays {} (no
fresh
871                         # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
872                         # deliberate cost of skipping the decode). Segmentation reads
neither of these.
873                         val_results = [
874                             ValidationResult(**r) for r in cached.get("results", [])
875                         ]
876                         reused_validation = True
877                         logger.info(
878                             "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
```

```
                            "validator config unchanged ? skipping the validator suite. "
                            "force=true re-validates.",
                            cached.get("passed"),
                            video_id,
                        )
                except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
                    reused_validation = False
                    logger.warning(
                        "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
                        video_id,
                        exc_info=True,
                    )
            if not reused_validation:
                logger.info(f"Running validations for {req.video_path}...")
                val_results, val_context = registry.run_all_with_context(local_video,
config)
            failures = [r for r in val_results if not r.passed]
            if not reused_validation:
                # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
                # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
                db.set_validation_cache(
                    video_id,
                    val_fingerprint,
                    not failures,
                    [r.model_dump() for r in val_results],
                )

            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
                )
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": _validation_failure_message(failures),
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db.add_video(
                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
            )

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
```

```
937                     "results": [r.model_dump() for r in val_results],
938                     "play_segments": [],
939                 }
940                 return response
941
942             logger.info(f"[API] Using segmenter: {seg_name}")
943             notify(f"segmenting ({seg_name})")
944             segmenter_actual = seg_name
945             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
946             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
947             play_wm, cand_wm = db.segment_watermarks(video_id)
948             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
949             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
950             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
951             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
952             # they're dropped on the cloud path in this slice ? a documented follow-up.)
953             remote = _maybe_dispatch_remote(
954                 config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
955             )
956             if remote is not None:
957                 response, segmentation_ran = (
958                     remote  # `ran` = did the cloud job actually execute (report)
959                 )
960                 return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
961             # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
962             compute_actual = "in_process"
963             segmenter = segmenter_cls(db, config)
964             try:
965                 # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
966                 # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
967                 with _LOCAL_COMPUTE_LANE:
968                     result = segmenter.process_video(
969                         video_id, local_video, req.player_pool, req.max_frames
970                     )
971             except Exception:
972                 # A raising segmenter must not leave half-written rows: restore the pre-run
973                 # state (same destroy-after-confirm contract as the Failure branch), then
let
974                 # the job worker record the failure.
975                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
976                 raise
977             segmentation_ran = True
978
```

26. user (2026-07-08T06:29:29.994Z)

```
978
979             if isinstance(result, Failure):
980                 logger.error(f"[API] Segmenter failed: {result.message}")
981                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
982                 response = {
983                     "video_id": video_id,
984                     "status": "failed",
985                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
986                     "results": [r.model_dump() for r in val_results],
```

```
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = _main.emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=_report_config_for_input(config, req.video_path),
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
```

```
1041                segmenter_actual=segmenter_actual,
1042                was_reingest=was_reingest,
1043                segmentation_ran=segmentation_ran,
1044            )
1045            if paths and isinstance(response, dict):
1046                response["reports"] = paths
1047            storage.cleanup_acquired_local_input(
```

27. user (2026-07-08T06:29:34.568Z)

```
1047            storage.cleanup_acquired_local_input(
1048                req.video_path, local_video, getattr(config, "storage", None)
1049            )
1050
1051
1052    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1053        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1054
1055        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1056        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1057        to a no-op rather than blocking a job it cannot measure."""
1058        try:
```

28. assistant (2026-07-08T06:29:38.962Z)

Now let me read the test file and the config test change.

29. user (2026-07-08T06:29:39.789Z)

```
1       """#524 win #2 ? trust the submit-time ``video_id`` instead of re-hashing on the drain.
2
3       The chunked upload streams the whole-file MD5 while it uploads
(``backend/api/uploads.py``), and the
4       submit hook records that same md5 on the job row (``submit_time_video_id``, #484 ? one
gs:// metadata
5       stat, converted to the identical HEX form ``compute_video_id`` produces). So when the
drain fetches
6       the clip to scratch it already KNOWS the video_id and need not re-hash the (often multi-
GB) file.
7
8       These tests pin the behaviour: with a well-formed trusted id and the knob on (default),
the drain
9       SKIPS ``compute_video_id`` and uses the trusted id verbatim; with the id absent,
malformed, or the
10      knob off, it hashes exactly as before. A spy on ``compute_video_id`` (counting
invocations) is the
11      proof ? mirrors ``tests/test_validation_cache.py``'s stubbed-suite pattern (fully
offline, no decode).
12      """
13
14      import backend.main as m
15      from backend.config.models import AppConfig, DeploymentConfig
16      from backend.infrastructure.database import Database
17      from backend.orchestration import _looks_like_video_id
18      from backend.pipeline.validators.base import ValidationResult
19
20      _VALID_ID = "a" * 32  # a well-formed 32-char lowercase-hex video_id
21      _HASH_SENTINEL = "b" * 32  # what a real compute_video_id() would return (spied)
22
23
24      class _HashSpy:
25          """Stands in for ``compute_video_id`` and counts real invocations. A TRUST hit must
```

```
NOT call this
        ? ``calls == 0`` is the proof the drain skipped the re-hash."""

        def __init__(self, ret=_HASH_SENTINEL):
            self._ret = ret
            self.calls = 0

        def __call__(self, *a, **k):
            self.calls += 1
            return self._ret


    class _OkSeg:
        """A segmenter that writes one play row (validation is stubbed to pass, so it's
reached)."""

        def __init__(self, db_, cfg_):
            self.db = db_

        def process_video(self, vid, path, *a, **k):
            sid = self.db.add_play_segment(vid, 0.0, 5.0)
            return [
                {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}
            ]


    def _pass_validators(*a, **k):
        return [ValidationResult(validator_name="x", passed=True, message="ok",
details={})], {}


    def _setup(tmp_path, monkeypatch, *, cfg=None):
        """Wire module globals + a temp-file DB + a hash spy + stubbed validators/segmenter.
        Returns ``(hash_spy, video_path)``. A bare/local ``video_path`` makes
``acquire_local_input`` an
        identity (no fetch), so the ONLY video_id source is the trusted id or the spied
hash."""
        db = Database(str(tmp_path / "t.db"))
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg or AppConfig())
        spy = _HashSpy()
        monkeypatch.setattr(m, "compute_video_id", spy)
        monkeypatch.setattr(m.registry, "run_all_with_context", _pass_validators)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _OkSeg)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"some-bytes")
        return spy, str(video)


    def _process(video, *, trusted):
        # config resolves from m.global_config (set by _setup) via the backward-compat shim.
        return m._execute_processing(
            m.ProcessRequest(video_path=video, segmenter="motion"),
            trusted_video_id=trusted,
        )


    # --- _looks_like_video_id (the format guard)
-------------------------------------------------------


    def test_looks_like_video_id_accepts_only_32_char_lowercase_hex():
        assert _looks_like_video_id("a" * 32)
        assert _looks_like_video_id("0123456789abcdef0123456789abcdef")
```

```
        # rejects: wrong length, uppercase, non-hex, non-str, empty, None
        assert not _looks_like_video_id("a" * 31)
        assert not _looks_like_video_id("a" * 33)
        assert not _looks_like_video_id("A" * 32)  # helper is strict-lowercase (callers
.lower() first)
        assert not _looks_like_video_id("g" * 32)
        assert not _looks_like_video_id("not-a-hash")
        assert not _looks_like_video_id("")
        assert not _looks_like_video_id(None)
        assert not _looks_like_video_id(12345678901234567890123456789012)  # int, not str


    # --- the drain: trust vs re-hash
------------------------------------------------------------------


    def test_trusts_valid_submit_id_and_skips_rehash(tmp_path, monkeypatch, caplog):
        """A well-formed trusted id + the default knob ? the drain reuses it and does NOT
call the hash."""
        spy, video = _setup(tmp_path, monkeypatch)
        with caplog.at_level("INFO", logger="backend.orchestration"):
            out = _process(video, trusted=_VALID_ID)
        assert out["video_id"] == _VALID_ID
        assert spy.calls == 0  # the whole point: no re-hash
        assert any("trusting submit-time video_id" in r.message for r in caplog.records)


    def test_hashes_when_no_trusted_id(tmp_path, monkeypatch):
        """No trusted id (local upload / non-gcs URI ? submit_time_video_id was None) ? hash
as before."""
        spy, video = _setup(tmp_path, monkeypatch)
        out = _process(video, trusted=None)
        assert out["video_id"] == _HASH_SENTINEL
        assert spy.calls == 1


    def test_ignores_trusted_id_when_knob_off(tmp_path, monkeypatch):
        """deployment.trust_submit_video_id=False forces the re-hash even with a valid
trusted id
        (belt-and-suspenders rollback, no redeploy)."""
        cfg = AppConfig(deployment=DeploymentConfig(trust_submit_video_id=False))
        spy, video = _setup(tmp_path, monkeypatch, cfg=cfg)
        out = _process(video, trusted=_VALID_ID)
        assert out["video_id"] == _HASH_SENTINEL
        assert spy.calls == 1


    def test_ignores_malformed_trusted_id_and_falls_back_to_hash(tmp_path, monkeypatch):
        """A malformed / wrong-length / wrong-encoding trusted id can never become the DB
key ?
        it degrades to a real hash."""
        spy, video = _setup(tmp_path, monkeypatch)
        for bad in ("not-a-valid-id", "a" * 31, "A" * 32, ""):
            spy.calls = 0
            out = _process(video, trusted=bad)
            assert out["video_id"] == _HASH_SENTINEL, bad
            assert spy.calls == 1, bad


    # --- the wiring: the drain hands _execute_processing the job row's video_id
-------------------------


    def test_run_claimed_job_threads_job_video_id_as_trusted(tmp_path, monkeypatch):
        """``_run_claimed_job`` must pass the job row's persisted video_id
```

```
(submit_time_video_id, #484) to
142         ``_execute_processing`` as ``trusted_video_id`` ? the wiring that makes win #2 fire
on the drain."""
143         db = Database(str(tmp_path / "jobs.db"))
144         monkeypatch.setattr(m, "db_instance", db)
145         monkeypatch.setattr(m, "global_config", AppConfig())
146         seen = {}
147
148         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
149             seen["trusted"] = trusted_video_id
150             return {"status": "success", "video_id": _VALID_ID, "play_segments": []}
151
152         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
153         db.create_job("pj", "gs://b/clip.mp4")
154         db.set_job_video_id("pj", _VALID_ID)
155         m._run_claimed_job(db.get_job("pj"), m.global_config, m.db_instance)
156         assert seen["trusted"] == _VALID_ID
157
158
159     def test_run_claimed_job_passes_none_when_job_has_no_video_id(tmp_path, monkeypatch):
160         """A job with no known video_id (local upload) threads None ? the drain hashes
(fallback)."""
161         db = Database(str(tmp_path / "jobs.db"))
162         monkeypatch.setattr(m, "db_instance", db)
163         monkeypatch.setattr(m, "global_config", AppConfig())
164         seen = {}
165
166         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
167             seen["trusted"] = trusted_video_id
168             return {"status": "success", "video_id": "x", "play_segments": []}
169
170         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
171         db.create_job("pj2", "/local/v.mp4")  # no video_id set
172         m._run_claimed_job(db.get_job("pj2"), m.global_config, m.db_instance)
173         assert seen["trusted"] is None
174
```

30. assistant (2026-07-08T06:29:50.399Z)

Let me check the other job-creation path `create_job` and its callers to ensure no client-supplied video_id can reach `jobs.video_id`.

31. user (2026-07-08T06:29:54.349Z)

```
backend/infrastructure/database_jobs.py:23:    def create_job(
```

32. user (2026-07-08T06:29:58.159Z)

```
23          def create_job(
24              self,
25              job_id: str,
26              video_path: str,
27              segmenter: Optional[str] = None,
28              player_pool: Optional[List[str]] = None,
29              max_frames: Optional[int] = None,
30              policy: Optional[Dict[str, Any]] = None,
31              owner_id: Optional[str] = None,
32              locale: Optional[str] = None,
33              compute: Optional[str] = None,
34              force: bool = False,
35          ) -> bool:
36              """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
37              (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
```

```
``owner_id``
38              (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
39              ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
40              ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
41              honours; NULL ? the server default (byte-identical to pre-picker behaviour).
``force`` opts
42              into deliberate reprocessing; false leaves params NULL for old-row
compatibility."""
43              params = json.dumps({"force": True}) if force else None
44              return self._execute_write(
45                  "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
46                  "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?,
?, ?, ?)",
47                  (
48                      job_id,
49                      video_path,
50                      segmenter,
51                      json.dumps(player_pool) if player_pool is not None else None,
52                      max_frames,
53                      json.dumps(policy) if policy is not None else None,
54                      owner_id,
55                      locale,
56                      compute,
57                      params,
58                  ),
59                  f"Failed to create job {job_id}",
60              )
61
62          def create_compile_job(
63              self,
64              job_id: str,
65              video_id: str,
66              video_path: str,
67              segment_ids: List[int],
```

33. user (2026-07-08T06:30:10.249Z)

```
143                 f"Failed to mark job {job_id} failed",
144             )
145
146         def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
147             """Fetch one job; `result`/`player_pool` JSON deserialized."""
148             with self._connect() as conn:
149                 row = (
150                     conn.cursor()
151                     .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
152                     .fetchone()
153                 )
154                 if not row:
155                     return None
156                 d = dict(row)
157                 d["result"] = json.loads(d["result"]) if d["result"] else None
158                 d["player_pool"] = (
159                     json.loads(d["player_pool"]) if d["player_pool"] else None
160                 )
161                 d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
162                 return d
163
164         def list_jobs(
165             self,
166             owner_id: Optional[str] = None,
```

```
167             statuses: Optional[List[str]] = None,
168             limit: int = 50,
169         ) -> List[Dict[str, Any]]:
170             """List a tenant's jobs, newest first ? the /api/jobs processing-queue view so a
client can
171             discover in-flight work it didn't submit (a new tab/device, cleared storage).
``owner_id``
172             NULL ? the local single-user's NULL-owner rows; a value ? ONLY that tenant's
rows (M9, D9
173             isolation ? never leak another tenant's jobs). ``statuses`` filters execution
state (e.g.
174             ['queued','running','waiting'] for the active queue); None ? any state. Light
projection: the
175             heavy ``result`` JSON is reduced to ``result_status`` (the pipeline verdict) so
the queue stays
176             cheap to poll."""
177             owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
178             params: List[Any] = [] if owner_id is None else [owner_id]
179             clauses = [owner_clause]
180             if statuses:
181                 clauses.append("status IN (%s)" % ",".join("?" for _ in statuses))
182                 params.extend(statuses)
183             sql = (
184                 "SELECT id, kind, status, progress, video_id, video_path, compute, error,
result, "
185                 "created_at, started_at, finished_at FROM jobs WHERE "
186                 + " AND ".join(clauses)
187                 + " ORDER BY created_at DESC, id DESC LIMIT ?"
188             )
189             params.append(int(limit))
190             with self._connect() as conn:
191                 rows = conn.cursor().execute(sql, tuple(params)).fetchall()
192             out: List[Dict[str, Any]] = []
193             for row in rows:
194                 d = dict(row)
195                 res = json.loads(d["result"]) if d.get("result") else None
196                 d["result_status"] = (res or {}).get("status") if res else None
197                 d.pop("result", None)
198                 out.append(d)
199             return out
200
201         def find_active_process_job(
202             self, owner_id: Optional[str], video_path: str
203         ) -> Optional[Dict[str, Any]]:
204             """Dedup key for /api/process idempotency: the {id, status} of an already-ACTIVE
205             (queued|running|waiting) PROCESS job for the same (owner_id, video_path), else
None ? so a
206             re-submit of the same video (a tester who can't see their running job) returns
that job
207             instead of dispatching a second, PAID GPU run. Compile jobs (kind='compile') are
excluded;
208             the owner scoping mirrors ``list_jobs`` (NULL ? local single-user)."""
209             owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
210             params: Tuple[Any, ...] = (
211                 (video_path,) if owner_id is None else (owner_id, video_path)
212             )
213             with self._connect() as conn:
214                 row = (
215                     conn.cursor()
216                     .execute(
217                         "SELECT id, status FROM jobs WHERE "
```

34. assistant (2026-07-08T06:30:13.759Z)

Now let me check the CLI path and all other callers of `_execute_processing`.

35. user (2026-07-08T06:30:20.410Z)

```
backend/api/job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
`_execute_processing`,
backend/api/job_worker.py:109:    is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend/api/job_worker.py:137:        # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
backend/api/job_worker.py:171:            result = _main._execute_processing(
backend/main.py:231:# backend.main ? `import backend.main as m; m._execute_processing(...)`, the
job worker's
backend/main.py:238:    _execute_processing,
backend/orchestration.py:3:The full hash -> validate -> segment -> report pipeline
(``_execute_processing``), the
backend/orchestration.py:13:...)`` and ``import backend.main as m; m._execute_processing(...)``
keep working unchanged.
backend/orchestration.py:722:def _execute_processing(
backend/orchestration.py:730:    test code that calls ``_execute_processing(req)`` still works.
backend/orchestration.py:746:    # Backward-compatible shim: if called as
_execute_processing(req) (old pattern),
backend/orchestration.py:749:        # Old signature: _execute_processing(req, progress=None)
```

36. user (2026-07-08T06:30:30.944Z)

```
209:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
210:# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
218:    _drain_due_jobs,
222:    _run_claimed_job,
223:    _schedule_next_drain_if_waiting,
224:    configure_drain,
231:# backend.main ? `import backend.main as m; m._execute_processing(...)`, the job worker's
238:    _execute_processing,
905:def process_video(req: ProcessRequest, request: Request):
950:    # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
964:    outcome = jobs_api.submit_process_job(
972:            _drain_due_jobs, request.app.state.config, request.app.state.db
1140:        _drain_due_jobs, request.app.state.config, request.app.state.db
1552:                result = segmenter.process_video(
1635:        configure_drain(global_config)  # drain concurrency from config before the executor
exists (#466)
1636:        _get_executor().submit(_drain_due_jobs, global_config, db_instance)
```

37. user (2026-07-08T06:30:41.102Z)

```
(Bash completed with no output)
```

38. assistant (2026-07-08T06:31:22.951Z)

Let me verify the test file passes and run lint/type checks on the changed code.

39. user (2026-07-08T06:31:46.511Z)

```
.......                                                                  [100%]
7 passed in 11.78s
```

40. user (2026-07-08T06:32:07.204Z)

```
All checks passed!
---MYPY---
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
```

```
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

41. assistant (2026-07-08T06:33:52.313Z)

I've completed a thorough trace of the win #2 trust path and its wiring. Let me do one final check on the `probe_process_cache` to confirm id consistency.

42. user (2026-07-08T06:33:57.385Z)

```
301:def probe_process_cache(db, cfg, video_id: str) -> bool:
334:def cached_process_result(
868:                cached = db.get_validation_cache(video_id)
898:            db.set_validation_cache(
```

43. user (2026-07-08T06:34:00.443Z)

```
301     def probe_process_cache(db, cfg, video_id: str) -> bool:
302         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
303
304         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
305         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
306         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
307         errors report False (? the normal dispatch path with its guards)."""
308         try:
309             existing = db.get_video(video_id)
310             if (
311                 existing
312                 and existing.get("status") == "processed"
313                 and db.query_play_segments(video_id, {})
314             ):
315                 return True
316             sc = _storage_settings(cfg)
317             if not sc or (sc.get("backend") or "local") == "local":
318                 return False
319             output_root = (sc.get("output_root") or "").rstrip("/")
320             if not output_root:
321                 return False
322             report_uri = f"{output_root}/outputs/{video_id}/report.json"
323             if not storage.exists(report_uri, config=sc):
324                 return False
325             report = _read_bucket_json(report_uri, sc)
326             return bool(report and report.get("status") == "success")
327         except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
328             logger.warning(
329                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
330             )
331             return False
332
333
334     def cached_process_result(
```

44. assistant (2026-07-08T06:34:38.391Z)

I've completed a thorough trace of the win #2 trust path and wiring. Here is my assessment.

Verification summary

Exception-cleanup in `_execute_processing` (orchestration.py:770-803) ? Correct. Every statement after `acquire_local_input` is inside the `try`. On the trust path `video_id = candidate` cannot throw and `logger.info` errors are swallowed by the logging framework (and would still be inside the `try`); on the re-hash path `compute_video_id` throwing is caught. Both branches hit `cleanup_acquired_local_input` + `raise`. No path leaves `local_video` acquired without cleanup. The end-of-function `finally` (line 1047) covers the happy path.

`_looks_like_video_id` (orchestration.py:242-253) ? Correct for every input: None/int ? `isinstance` False; ""/31/33 ? length gate; uppercase "A"*32 ? not in lowercase hexset; non-hex "g" ? rejected. It exactly matches what `submit_time_video_id` can emit: `gcs.py::_md5_hex` returns `raw.hex()` (lowercase) only when `len(raw)==16` (? 32 chars), else None. So every real submit-time id passes and no spurious re-hash occurs. `compute_video_id` = `hexdigest()` (lowercase 32) matches too.

Authoritativeness of `job.get("video_id")` (job_worker.py:180) ? Confirmed server-derived only. `ProcessRequest` (main.py:258-272) has NO `video_id` field, so the `/api/process` body cannot inject one. The only production writer of `jobs.video_id` for a process job is `claim_or_create_process_job(video_id=?)`, fed from `submit_process_job` ? `submit_time_video_id(input_ref, cfg)` (gs:// metadata stat, server-side). `create_job` (database_jobs.py:23) doesn't even insert a video_id column. `get_job` = `SELECT *` so the dict carries the real column. `... or None` correctly maps `""` ? None ? re-hash.

Divergence guard (orchestration.py:661) ? Not weakened. The worker still independently re-hashes the bucket bytes; the control plane does not forward the trusted id to the worker, so the guard remains a genuine cross-check. Trusting the submit-time id widens the "changed input" detection window (submit?worker instead of drain-fetch?worker) rather than masking it. Warning-only; no functional break.

#518 validation cache + `probe_process_cache` (orchestration.py:301, 868, 898) ? video_id stays byte-identical. Both the submit-time probe and (now) the drain use `submit_time_video_id`, so the cache/probe keys are *more* coherent than before, not less. For composite objects `_md5_hex`?None?trust skipped?re-hash, matching pre-#524 exactly.

Other callers ? `_execute_processing` has exactly one production caller (job_worker.py:171); every other invocation (CLI/tests via the back-compat shim) defaults `trusted_video_id=None` ? re-hash. No regression.

Gates: new tests pass (7/7), `ruff` clean, `mypy` clean on the changed files.

The one residual behavior (a gs:// object overwritten *between submit and drain* would key results under the submit-time id on the local path with no worker cross-check) is a pre-existing race ? pre-#524 the submit-probe id and the drain re-hash already diverged in that case ? and it is explicitly inside the documented trust trade-off with a rollback knob. Not a defect introduced by this PR.

45. user (2026-07-08T06:34:40.116Z)

Structured output provided successfully